

# CO1 AT1 — Descriptive Experiment Exam

## MLA0304 — Reinforcement Learning for Environment and Agent

### Code of Conduct:

I **S.CHANDRA SIDDHARDHA (192525222)** (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

S.siddhardha

**Signature of the student:**

### Course Details

**Course Name:** MLA03 **Course Code:** Reinforcement Learning for Environment and Agent

**Course Outcome:** CO1 — Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3) **Assessment:** Descriptive Experiment Exam — 10 Questions, 50 Marks

## Experiment 1: Installation of Reinforcement Learning Development Environment

**Aim:** To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium, NumPy, Matplotlib, and verify the environment.

### Theory

A Reinforcement Learning (RL) development environment needs three layers of software: (1) a Python distribution and package manager (Anaconda/Conda), (2) numerical and plotting libraries (NumPy, Matplotlib), and (3) RL-specific and deep-learning libraries (Gymnasium, TensorFlow, Keras). Setting these up correctly avoids version-conflict errors later, since TensorFlow and Gymnasium both depend on specific NumPy versions.

## Procedure

1. Download and install **Anaconda Navigator** from [anaconda.com](https://anaconda.com).
2. Open **Anaconda Prompt** and create an isolated environment so packages don't conflict with the system Python:

```
conda create -n rl_env python=3.10 -y
conda activate rl_env
```

3. Install the core libraries:

```
pip install numpy matplotlib gymnasium tensorflow keras
```

4. Verify installation by printing library versions:

```
import numpy, matplotlib, gymnasium, tensorflow, keras
```

```
print("NumPy:", numpy.__version__)
print("Matplotlib:", matplotlib.__version__)
print("Gymnasium:", gymnasium.__version__)
print("TensorFlow:", tensorflow.__version__)
print("Keras:", keras.__version__)
```

5. Run a minimal RL program to confirm the environment works end-to-end:

```
import gymnasium as gym
```

```
env = gym.make("CartPole-v1")
state, info = env.reset()
```

```
for step in range(10):
    action = env.action_space.sample()      # random action
    next_state, reward, terminated, truncated, info = env.step(action)
    print(f"Step {step}: action={action}, reward={reward}, done={terminated or truncated}")
    if terminated or truncated:
        state, info = env.reset()
```

```
env.close()
```

## Expected Output

The version-check script prints the installed version numbers with no import errors. The CartPole script prints 10 lines showing the action taken, the reward received (+1 per step for CartPole), and whether the episode ended (pole fell or cart left bounds).

## Result

The Reinforcement Learning development environment was successfully installed and verified. All the required libraries (NumPy, Matplotlib, Gymnasium, TensorFlow, Keras) were confirmed

working through a sample CartPole interaction, establishing the base setup needed for the remaining experiments.

## Experiment 2: Familiarization with OpenAI Gym / Gymnasium Environments

**Aim:** To create, initialize, and interact with FrozenLake, CartPole, and MountainCar to understand states, actions, rewards, and episodes.

### Theory

Every Gymnasium environment exposes a common interface:

- **reset()** — starts a new episode and returns the initial state (observation).
- **step(action)** — applies an action and returns (next\_state, reward, terminated, truncated, info).
- **observation\_space** — the set of possible states.
- **action\_space** — the set of possible actions.

An **episode** is one complete run from the initial state to a terminal state (goal reached, failure, or time limit).

### Code

```
import gymnasium as gym
```

```
def explore_env(env_name, episodes=1, max_steps=20):
    env = gym.make(env_name)
    print(f"\n--- {env_name} ---")
    print("Observation space:", env.observation_space)
    print("Action space:", env.action_space)

    for ep in range(episodes):
        state, info = env.reset()
        total_reward = 0
        for t in range(max_steps):
            action = env.action_space.sample()
            state, reward, terminated, truncated, info = env.step(action)
            total_reward += reward
            if terminated or truncated:
                break
        print(f"Episode {ep+1}: steps={t+1}, total_reward={total_reward}")
    env.close()
```

```
explore_env("FrozenLake-v1")
explore_env("CartPole-v1")
explore_env("MountainCar-v0")
```

## Observations

| Environment | States                                                                      | Actions                            | Reward Signal                                 |
|-------------|-----------------------------------------------------------------------------|------------------------------------|-----------------------------------------------|
| FrozenLake  | 16 discrete tiles (4×4 grid)                                                | 4 (Left, Down, Right, Up)          | +1 on reaching goal, 0 otherwise              |
| CartPole    | 4 continuous values (cart position, velocity, pole angle, angular velocity) | 2 (push left, push right)          | +1 for every timestep the pole stays balanced |
| MountainCar | 2 continuous values (position, velocity)                                    | 3 (push left, no push, push right) | −1 per timestep until the goal is reached     |

## Result

FrozenLake, CartPole, and MountainCar were successfully created and explored using Gymnasium. Each environment's state space, action space, reward structure, and episode termination behavior were observed and tabulated, confirming a clear understanding of the standard RL interaction loop.

## Experiment 3: Implementation of the Reinforcement Learning Framework

**Aim:** To implement the basic RL framework — an agent interacting with an environment through states, actions, rewards, and policies.

## Theory

The RL framework is a closed loop:

Agent --action--> Environment

Agent <--(state, reward)-- Environment

At each timestep  $t$ , the agent observes state  $s_t$ , selects an action  $a_t$  using a **policy**  $\pi(a|s)$ , receives reward  $r_t$ , and transitions to state  $s_{t+1}$ . The goal of the agent is to learn a policy that maximizes cumulative reward.

## Code

```
import gymnasium as gym
import numpy as np

class RandomPolicyAgent:
    """A minimal agent implementing the standard RL loop."""
    def __init__(self, action_space):
        self.action_space = action_space

    def select_action(self, state):
        # policy:  $\pi(a|s)$  -- here, a uniform random policy
        return self.action_space.sample()

def run_episode(env, agent):
    state, info = env.reset()
    total_reward = 0
    done = False
    trajectory = []

    while not done:
        action = agent.select_action(state)
        next_state, reward, terminated, truncated, info = env.step(action)
        trajectory.append((state, action, reward))
        total_reward += reward
        state = next_state
        done = terminated or truncated

    return total_reward, trajectory

env = gym.make("CartPole-v1")
agent = RandomPolicyAgent(env.action_space)

rewards = []
for ep in range(5):
    total_reward, traj = run_episode(env, agent)
    rewards.append(total_reward)
    print(f"Episode {ep+1}: total_reward={total_reward}, steps={len(traj)}")

print("Average reward over 5 episodes:", np.mean(rewards))
env.close()
```

## Result

A generic Agent–Environment interaction framework was implemented, in which the agent selects actions according to a policy, receives rewards from the environment, and the trajectory of (state, action, reward) is logged for each episode. This confirms the fundamental structure that underlies every RL algorithm.

## Experiment 4: Simulation of a Markov Decision Process (MDP)

**Aim:** To develop a simple MDP model and simulate state transitions, reward functions, and policy execution.

### Theory

An MDP is formally the tuple  $(\mathbf{S}, \mathbf{A}, \mathbf{P}, \mathbf{R}, \gamma)$ :

- $\mathbf{S}$  — set of states
- $\mathbf{A}$  — set of actions
- $\mathbf{P}(\mathbf{s}'|\mathbf{s},\mathbf{a})$  — transition probability
- $\mathbf{R}(\mathbf{s},\mathbf{a},\mathbf{s}')$  — reward function
- $\gamma$  — discount factor

The **Markov property** states that the next state depends only on the current state and action, not on the history.

### Code — A 3-State MDP

```
import numpy as np
```

```
states = ["Start", "Middle", "Goal"]
```

```
actions = ["Forward", "Stay"]
```

```
# Transition probabilities  $P[\text{state}][\text{action}] = \{\text{next\_state}: \text{probability}\}$ 
```

```
P = {
```

```
    "Start": {"Forward": {"Middle": 0.8, "Start": 0.2},
```

```
            "Stay": {"Start": 1.0}},
```

```
    "Middle": {"Forward": {"Goal": 0.7, "Middle": 0.3},
```

```
            "Stay": {"Middle": 1.0}},
```

```
    "Goal": {"Forward": {"Goal": 1.0},
```

```
            "Stay": {"Goal": 1.0}},
```

```
}
```

```

# Reward function  $R(s, a)$ 
R = {
    "Start": {"Forward": -1, "Stay": -1},
    "Middle": {"Forward": -1, "Stay": -1},
    "Goal": {"Forward": 10, "Stay": 0},
}

policy = {"Start": "Forward", "Middle": "Forward", "Goal": "Stay"}

def simulate(start_state, steps=10, gamma=0.9):
    state = start_state
    total_return, discount = 0, 1
    for t in range(steps):
        action = policy[state]
        next_states = list(P[state][action].keys())
        probs = list(P[state][action].values())
        reward = R[state][action]
        total_return += discount * reward
        print(f't={t}: state={state}, action={action}, reward={reward}')
        state = np.random.choice(next_states, p=probs)
        discount *= gamma
        if state == "Goal" and t > 0:
            print(f't={t+1}: state=Goal (terminal)')
            break
    return total_return

ret = simulate("Start")
print("Discounted Return:", round(ret, 3))

```

## Result

A 3-state MDP (Start  $\rightarrow$  Middle  $\rightarrow$  Goal) was simulated with defined transition probabilities and rewards. Executing a fixed policy over several timesteps produced a discounted cumulative return, demonstrating how sequential decision-making unfolds under the Markov property.

## Experiment 5: Bellman Equation Demonstration

**Aim:** To implement the Bellman Expectation and Bellman Optimality Equations for state-value and action-value functions.

## Theory

**Bellman Expectation Equation** (for a fixed policy  $\pi$ ):

$$V\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [ R(s,a,s') + \gamma V\pi(s') ]$$

**Bellman Optimality Equation:**

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) [ R(s,a,s') + \gamma V^*(s') ]$$

These are solved iteratively (value iteration) until  $V(s)$  converges.

## Code — Value Iteration on the MDP from Experiment 4

```
import numpy as np
```

```
states = ["Start", "Middle", "Goal"]
```

```
actions = ["Forward", "Stay"]
```

```
gamma = 0.9
```

```
V = {s: 0.0 for s in states}
```

```
def bellman_optimality_update(V):
```

```
    new_V = {}
```

```
    for s in states:
```

```
        action_values = []
```

```
        for a in actions:
```

```
            value = 0
```

```
            for s_next, prob in P[s][a].items():
```

```
                value += prob * (R[s][a] + gamma * V[s_next])
```

```
            action_values.append(value)
```

```
        new_V[s] = max(action_values)
```

```
    return new_V
```

```
print("Iteration | Start | Middle | Goal")
```

```
for i in range(15):
```

```
    V = bellman_optimality_update(V)
```

```
    print(f"{i:9d} | {V['Start']:.3f} | {V['Middle']:.3f} | {V['Goal']:.3f}")
```

```
# Extract the optimal policy from the converged values
```

```
optimal_policy = {}
```

```
for s in states:
```

```
    best_a, best_val = None, -np.inf
```

```
    for a in actions:
```

```
        val = sum(p * (R[s][a] + gamma * V[s_next]) for s_next, p in P[s][a].items())
```

```
        if val > best_val:
```

```

        best_val, best_a = val, a
    optimal_policy[s] = best_a

print("\nOptimal Policy:", optimal_policy)
print("Optimal Values:", V)

(P and R are the dictionaries defined in Experiment 4.)

```

## Result

The Bellman Expectation and Optimality equations were implemented through iterative value updates. The value function was observed to converge after a small number of iterations, and the corresponding optimal policy (always choosing “Forward”) was correctly extracted — confirming the convergence property of dynamic-programming-based value estimation.

## Experiment 6: Exploration vs. Exploitation Strategies

**Aim:** To implement  $\epsilon$ -Greedy, Greedy, and Random strategies and compare their performance.

### Theory

- **Greedy:** always picks the action with the highest known estimated value — exploits, but risks getting stuck on a suboptimal action found early.
- **Random:** always picks a random action — explores fully, but never exploits what has been learned.
- **$\epsilon$ -Greedy:** picks the best-known action with probability  $(1-\epsilon)$ , and a random action with probability  $\epsilon$  — balances the two.

### Code

```

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
true_means = [1.0, 1.5, 2.0, 0.5, 1.8]  # true reward mean per action
n_actions = len(true_means)
n_steps = 1000

def run_strategy(strategy, epsilon=0.1):
    Q = np.zeros(n_actions)
    N = np.zeros(n_actions)
    rewards = []
    for t in range(n_steps):

```

```

if strategy == "greedy":
    action = np.argmax(Q)
elif strategy == "random":
    action = np.random.randint(n_actions)
elif strategy == "epsilon_greedy":
    action = np.random.randint(n_actions) if np.random.rand() < epsilon else np.argmax(Q)
reward = np.random.normal(true_means[action], 1.0)
N[action] += 1
Q[action] += (reward - Q[action]) / N[action]
rewards.append(reward)
return np.cumsum(rewards) / (np.arange(n_steps) + 1)

greedy_curve = run_strategy("greedy")
random_curve = run_strategy("random")
eps_curve = run_strategy("epsilon_greedy", epsilon=0.1)

plt.plot(greedy_curve, label="Greedy")
plt.plot(random_curve, label="Random")
plt.plot(eps_curve, label="Epsilon-Greedy (0.1)")
plt.xlabel("Steps"); plt.ylabel("Average Reward"); plt.legend()
plt.title("Exploration vs Exploitation Strategies")
plt.savefig("strategy_comparison.png")
plt.show()

print("Final average reward — Greedy:", round(greedy_curve[-1], 3))
print("Final average reward — Random:", round(random_curve[-1], 3))
print("Final average reward — Epsilon-Greedy:", round(eps_curve[-1], 3))

```

## Observation

Random consistently earns the lowest average reward because it never exploits. Greedy converges fast but can lock onto a suboptimal action if its early estimate was lucky.  $\epsilon$ -Greedy achieves the highest long-run average reward because it keeps exploring a little while mostly exploiting the best-known action.

## Result

The three action-selection strategies were implemented and compared over 1000 steps.  $\epsilon$ -Greedy outperformed both pure Greedy and pure Random, empirically demonstrating why a balance of exploration and exploitation is essential in Reinforcement Learning.

## Experiment 7: Multi-Armed Bandit Problem

**Aim:** To implement the Multi-Armed Bandit problem using  $\epsilon$ -Greedy and UCB and evaluate cumulative rewards.

### Theory

The Multi-Armed Bandit is the simplest RL setting: one state,  $k$  actions (“arms”), each with an unknown reward distribution. The goal is to maximize cumulative reward over a fixed number of pulls.

**UCB (Upper Confidence Bound)** selects the action maximizing:

$$Q(a) + c \sqrt{(\ln(t) / N(a))}$$

which naturally favors actions that are either high-value or under-explored (small  $N(a)$ ).

### Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(1)
true_means = [1.0, 1.5, 2.0, 0.5, 1.8]
n_actions = len(true_means)
n_steps = 2000

def epsilon_greedy_bandit(epsilon=0.1):
    Q, N = np.zeros(n_actions), np.zeros(n_actions)
    rewards = []
    for t in range(n_steps):
        a = np.random.randint(n_actions) if np.random.rand() < epsilon else np.argmax(Q)
        r = np.random.normal(true_means[a], 1.0)
        N[a] += 1
        Q[a] += (r - Q[a]) / N[a]
        rewards.append(r)
    return np.cumsum(rewards)

def ucb_bandit(c=2.0):
    Q, N = np.zeros(n_actions), np.zeros(n_actions)
    rewards = []
    for t in range(1, n_steps + 1):
        if 0 in N:
            a = np.argmin(N)      # pull each arm once first
        else:
            ucb_values = Q + c * np.sqrt(np.log(t) / N)
```

```

        a = np.argmax(ucb_values)
        r = np.random.normal(true_means[a], 1.0)
        N[a] += 1
        Q[a] += (r - Q[a]) / N[a]
        rewards.append(r)
    return np.cumsum(rewards)

eps_cum = epsilon_greedy_bandit()
ucb_cum = ucb_bandit()

plt.plot(eps_cum, label="Epsilon-Greedy")
plt.plot(ucb_cum, label="UCB")
plt.xlabel("Trials"); plt.ylabel("Cumulative Reward"); plt.legend()
plt.title("Multi-Armed Bandit: Epsilon-Greedy vs UCB")
plt.savefig("bandit_comparison.png")

print("Total reward — Epsilon-Greedy:", round(eps_cum[-1], 2))
print("Total reward — UCB:", round(ucb_cum[-1], 2))

```

## Result

Both  $\epsilon$ -Greedy and UCB algorithms were implemented for the Multi-Armed Bandit problem. UCB typically achieved a higher cumulative reward than  $\epsilon$ -Greedy because it explores systematically (favoring under-tried arms) rather than randomly, converging faster to the best arm (arm index 2, mean = 2.0).

## Experiment 8: Reward Visualization in Reinforcement Learning

**Aim:** To simulate an RL agent and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib.

### Code

```

import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

env = gym.make("CartPole-v1")
n_episodes = 100
episode_rewards = []

for ep in range(n_episodes):

```

```

state, info = env.reset()
total_reward = 0
done = False
while not done:
    action = env.action_space.sample()    # random policy for demonstration
    state, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    done = terminated or truncated
    episode_rewards.append(total_reward)

env.close()

cumulative_rewards = np.cumsum(episode_rewards)
moving_avg = np.convolve(episode_rewards, np.ones(10)/10, mode="valid")

fig, axs = plt.subplots(1, 3, figsize=(15, 4))

axs[0].plot(episode_rewards)
axs[0].set_title("Reward per Episode")
axs[0].set_xlabel("Episode"); axs[0].set_ylabel("Reward")

axs[1].plot(cumulative_rewards)
axs[1].set_title("Cumulative Reward")
axs[1].set_xlabel("Episode"); axs[1].set_ylabel("Cumulative Reward")

axs[2].plot(moving_avg)
axs[2].set_title("10-Episode Moving Average (Learning Trend)")
axs[2].set_xlabel("Episode"); axs[2].set_ylabel("Average Reward")

plt.tight_layout()
plt.savefig("reward_visualization.png")
plt.show()

```

## Result

Episode rewards, cumulative reward, and a moving-average learning curve were plotted for 100 CartPole episodes under a random policy. The visualizations make it easy to see reward variance per episode and to judge — for a learning agent — whether performance is trending upward over time.

## Experiment 9: TensorFlow and Keras-Based Neural Network for RL

**Aim:** To build a feed-forward neural network for approximating value functions.

### Theory

When the state space is continuous or too large for a lookup table, a neural network approximates  $V(s)$  or  $Q(s,a)$  instead. This is the basis of Deep Q-Networks (DQN). The network takes a state vector as input and outputs a value (or one value per action).

### Code

```
import tensorflow as tf
from tensorflow import keras
from keras import layers
import numpy as np
import gymnasium as gym

env = gym.make("CartPole-v1")
state_dim = env.observation_space.shape[0] # 4 for CartPole
n_actions = env.action_space.n           # 2 for CartPole

# Q-value approximator: state -> Q-value for each action
model = keras.Sequential([
    layers.Input(shape=(state_dim,)),
    layers.Dense(24, activation="relu"),
    layers.Dense(24, activation="relu"),
    layers.Dense(n_actions, activation="linear") # linear output = raw Q-values
])

model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001), loss="mse")
model.summary()

# Demonstration: predict Q-values for a batch of random states
sample_states = np.random.uniform(-1, 1, size=(5, state_dim))
q_values = model.predict(sample_states)
print("Predicted Q-values:\n", q_values)
print("Greedy actions:", np.argmax(q_values, axis=1))
```

## Result

A two-hidden-layer feed-forward neural network (24-24 units, ReLU activations) was built using TensorFlow/Keras to approximate  $Q(s,a)$  for CartPole's 4-dimensional state space. The (untrained) network correctly outputs one Q-value per action for each input state, confirming the network architecture is ready to be trained with a DQN-style update rule.

## Experiment 10: Mini Reinforcement Learning Project Using CartPole

**Aim:** To develop an RL agent using TensorFlow/Keras to solve CartPole and evaluate learning performance.

### Theory

This experiment combines Experiments 6, 8, and 9 into a simplified **DQN agent**: a neural Q-function,  $\epsilon$ -greedy action selection, an experience replay buffer, and target-based Q-learning updates.

### Code:

```
import numpy as np
import random
from collections import deque
import gymnasium as gym
import tensorflow as tf
from tensorflow import keras
from keras import layers

env = gym.make("CartPole-v1")
state_dim = env.observation_space.shape[0]
n_actions = env.action_space.n

def build_model():
    model = keras.Sequential([
        layers.Input(shape=(state_dim,)),
        layers.Dense(24, activation="relu"),
        layers.Dense(24, activation="relu"),
        layers.Dense(n_actions, activation="linear")
    ])
    model.compile(optimizer="adam", loss="mse")
    return model
```

```

model = build_model()
memory = deque(maxlen=2000)

gamma = 0.95
epsilon, epsilon_min, epsilon_decay = 1.0, 0.01, 0.995
batch_size = 32
n_episodes = 50
episode_rewards = []

for ep in range(n_episodes):
    state, info = env.reset()
    state = np.reshape(state, [1, state_dim])
    total_reward = 0
    done = False

    while not done:
        # epsilon-greedy action selection
        if np.random.rand() < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(model.predict(state, verbose=0)[0])

        next_state, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated
        next_state = np.reshape(next_state, [1, state_dim])
        memory.append((state, action, reward, next_state, done))
        state = next_state
        total_reward += reward

    # experience replay training
    if len(memory) >= batch_size:
        batch = random.sample(memory, batch_size)
        for s, a, r, s_next, d in batch:
            target = r if d else r + gamma * np.amax(model.predict(s_next, verbose=0)[0])
            target_f = model.predict(s, verbose=0)
            target_f[0][a] = target
            model.fit(s, target_f, epochs=1, verbose=0)

epsilon = max(epsilon_min, epsilon * epsilon_decay)
episode_rewards.append(total_reward)
print(f"Episode {ep+1}/{n_episodes}: reward={total_reward}, epsilon={epsilon:.3f}")

```

```
success_rate = np.mean(np.array(episode_rewards) >= 195) * 100
print(f"\nSuccess rate (reward >= 195): {success_rate:.1f}%")
print("Average reward over last 10 episodes:", np.mean(episode_rewards[-10:]))
```

*(Note: for speed on CPU, n\_episodes can be reduced; the full CartPole “solved” criterion is an average reward  $\geq 195$  over 100 consecutive episodes.)*

## Result

A mini DQN-based Reinforcement Learning agent was built and trained on the CartPole-v1 environment using a Keras neural network,  $\epsilon$ -greedy exploration with decay, and experience replay. Episode rewards were observed to increase over training, and the success rate metric confirmed the agent was progressively learning to balance the pole for longer durations — successfully completing the mini RL project.